ch 9. INPUT ENHANCEMENT

Input enhancement is an another algorithmic design technique which uses the concept of space & time trade-off.

9.1 General plan of input enhancement

Given a pblm to solve. Instead of solving the given pblm directly, first we do some operations on the given i/p. That means, we generate/acquire some additional/extra infor^n about the pblm/input.

With this additional infor^n, we can solve the pblm in a quicker/less time.

Since, we are generating additional infor^n, we need to store that extra infor^n. Thus, we are using some additional memory space to store that extra infor^n.

Thus, we can achieve time effecient alg at the cost of extra space.

Examples

1) Sorting algorithms
   - → Sorting by Comparison Counting
   - → Sorting by Distribution Counting

2) String matching algorithms
   - → Horspool alg
   - → Boyer-Moore alg
   - → Knuth-Morris-Pratt alg

3) Dimensionality Reduction with PCA (Principal Component Analysis)

High dimensional data causes slow processing & results in overfitting in ML

By Applying PCA, we can reduce the no of features by retaining most of the variance in the data.

With the additional/extra info, we can solve the given pblm.

9.2 Input enhancement in string matching

We already solve string matching alg using brute-force. In that, we are shifting the pattern always by 1 position towards it right whenever mismatch occurs.

That means, we are wasting the time in shifting the pattern many no of times unnecessarily.

To avoid @ reduce this unnecessary shifting's, we go for better string matching algorithms, such as

1) Boyer-Moore alg
2) Horspool alg
3) Knuth-Morris-Pratt alg.

Horspool algorithm

Actually, Horspool alg is an simplified version of Boyer-Moore. Boyer-Moore alg is simple but implementation is not simple. So, we go for its simplified version.

Using, Horspool alg, we can avoid unnecessary shifting of pattern towards its right.

Horspool alg makes very less no of jumps/shifts when Compared to Brute Force string matching alg.

Horspool alg generates additional info" about the characters which are present in the given pattern & text and store that additional info" in a table called shift table.

This shift table gives us the info" about how many characters we need to jump/shift the pattern when mismatch occurs.

In Horspool alg also, we are aligning the given pattern against the text from the beginning.

But, instead of comparing the characters of the pattern from the beginning, we start comparing from the last.

Ex: A B A A A B C D ⟶ Text

A B C ⟶ pattern.

A B A A A B C D
A B C
In Brute Force

A B A A A B C D
A B C
In Horspool.

Horspool alg decides how much distance, pattern has to be shifted when mismatch occurs depends on character 'c' of the text (which was mismatched).

In general, there will be 4 possibilities

- Case 1:- Mismatch occurs at 1st comparison only & there is no occurrence of 'c's (mismatch character of text) in the pattern.

Ex:- Text ⟶ t_0 t_1 t_2 ---- C ---- t_{n-1}

BARBER

In this ex, mismatch occurs at 1st comparison only. The mismatch character is not present in the remaining characters of the pattern.

In such case, shift the pattern by its entire length (pattern size).

Case 2 :- Mismatch occurs at 1st Comparison itself & mismatch character is present in the remaining character of the pattern.

Text: $t_0 t_1 t_2$ - - - B - - - - - - - - t_{n-1}

pattern: - BARBER

mismatch occurs, but 'B' is present in pattern.

In such a case, shift the pattern by mismatch character's shift value. (In this case, shift value of B).

$t_0 t_1 t_2$ - - - - - B - - - - - - - - t_{n-1}

BARBER

2 1

BARBER (Pattern is shifted 2 position ∵ shift value of B is 2)

MADHUSUDHAN M.V.
Dept. of CSE

Case 3 :- Mismatch occurs after few successful Comparison's & mismatch character not present in the remaining characters of pattern.

$t_0 t_1 t_2$ - - - - MER - - - - - - - - - t_{n-1}

LEADER

Mismatch occurs, & mismatch character 'M' is not present in the remaining characters of pattern.

In such a case, shift the pattern by pattern's length.

$t_0 t_1 t_2$ - - - - - - - MER - - - - - - - - t_{n-1}

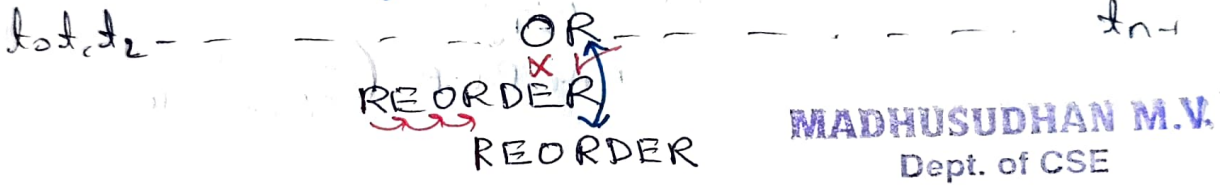
LEADER

LEADER

Case 4 :- Mismatch occurs after few successfull comparisons & mismatch character present in the remaining characters of the pattern

In 2nd comparison, mismatch occurs & mismatch character is present in the remaining characters of pattern & it (O) is at a distance '4' to the last character of pattern.

In this case, we are shifting the pattern 3 position because, abc already 1 character is matched So, 4-1=3



The shift table values are computed as follows:

Mismatch occurs at	Mismatch character (c) present in remaining character (P) or not	Shift value
1st time only	not present	pattern length (m)
1st time only	present	distance of the rightmost occurrence of c in pattern to the last character of pattern → d.
after few (x) successfull comparisons	not present	pattern length (m)
after few (x) successfull comparisons	present	d - 1

Shift table.

Ex(1) REORDER → pattern

pattern length = 7

character	Shift Value
R	3
E	1
O	4
D	2
others	7

//purpose :- To generate the shift table used by Horspool & Boyer-Moore alg

//Input :- pattern p[0...m-1] & an alphabet of all possible characters

//Output :- Table[0...size-1] indexed by alphabet's characters & filled with shift sizes. ©

```
Shift_Table (p[0...m-1])
{
    Initialize all the elements of Table[] with 'm'.
    for j ← 0 to m-2 do
        Table[p[j]] ← m-1-j
    end for
    return Table
}
```


Problem-1

Apply Horspool alg for the following pattern & text.

Text :- NOBODY_NOTICED_HIM

pattern NOT.

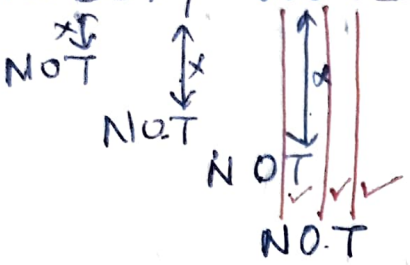
Solution pattern : p[0] H | p[1] O | p[2] T pattern size, m = 3

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Table	3	3	3	3	3	3	3	2	3	3	3	3	3	3	1	3	3	3	3	3	3	3	3	3	3	3

Shift table

H — 2	
O — 1	
others — 3	

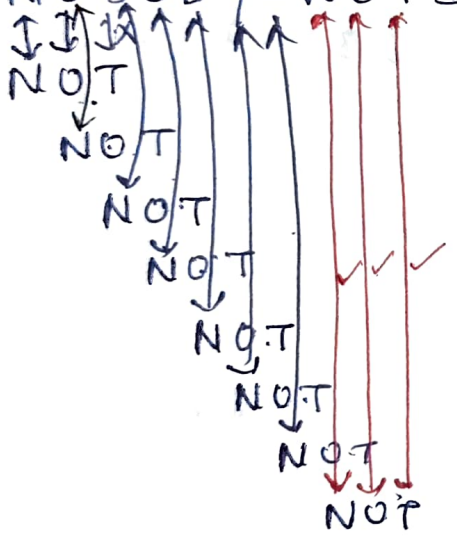
NOBODY - NOTICED - HIM.



→ Horspool

Here, we shifted, the pattern 3 times & we did totally 6 comparisons.

NOBODY - NOTICED - HIM



By applying brute force, we shifted 7 times & 12 comparisons which were much more than the Horspool.

Thus, we can say, Horspool alg is much more effecient than the brute force string matching alg.

Steps followed by Horspool alg are

1) For the given pattern of length 'm' & for all other alphabets, Construct the shift table.
2) Align the pattern against the text from the beginning.
3) Repeat the following until either pattern is found @ the pattern reaches beyond the last character of text.

→ Start comparing the last character of pattern with the corresponding character of text until all 'm' characters are matched (in this case, stop searching) @ mismatch occurs

→ If mismatch occurs, get the shift value of the corerponding mismatched character of text from the shift table. & shift the pattern towards its right by that value (shift value).

MADHUSUDHAN M.V.
Dept. of CSE

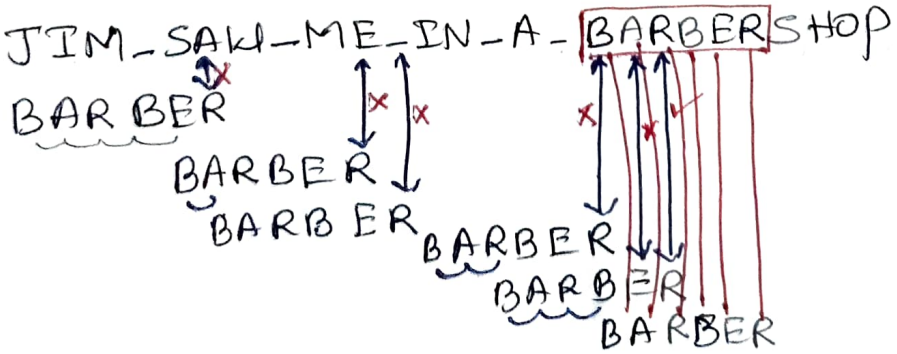
Problem (2)

Text: JIM_SAW_ME_IN_A_BARBERSHOP

Pattern: BARBER

Solution:-

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
4	2	6	6	1	6	6	6	6	6	6	6	6	6	6	6	6	3	6	6	6	6	6	6	6	6



Actually, here, 'A' value is 4 but we shifted 3 position ∵ already one character i.e., R is matched. So 4-1 = (3)

5 shifts & 12 Comparisons → Horspool

16 shifts & 22 Comparisons → Brute force

//Purpose :- Implementing string matching using Horspool's alg

//Input :- Text T[0...n-1] of n characters &
Pattern P[0...m-1] of m characters. $m \leq n$.

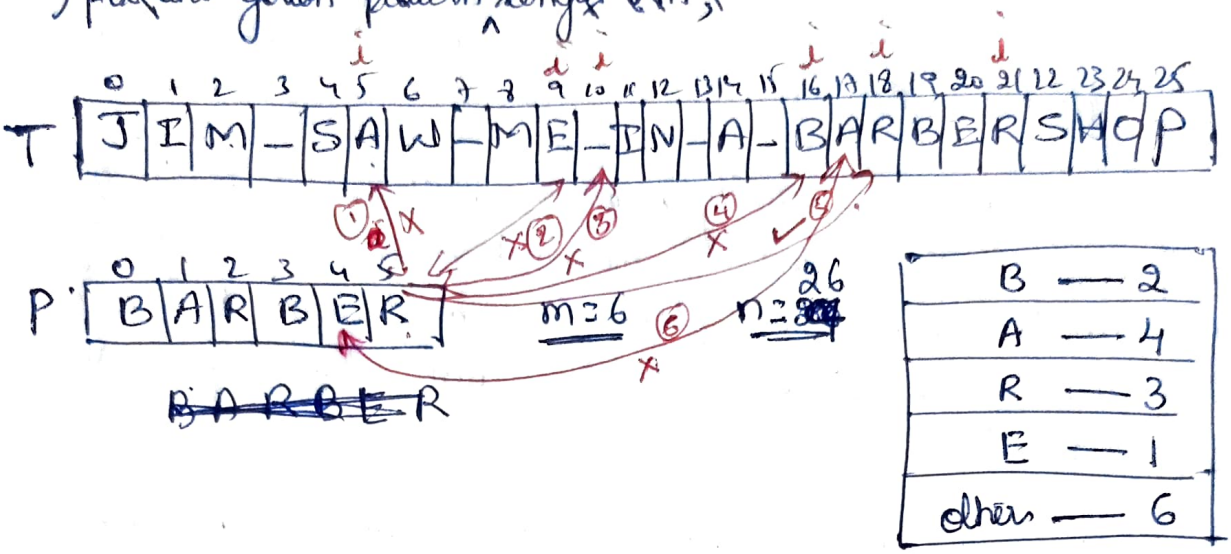
//Output :- Index of the left end of the 1st matching substring
otherwise returns -1.

ALGORITHM:

```
Horspool(P[0...m-1], T[0...n-1])
{
    Shift_Table(P[0...m-1]) --> Calls Shift-Table() func to generate shift values
    i ← m-1 ——→ index for pattern (point to last char, initially)
    while i ≤ n-1 do
        k ← 0                    Initially matched characters = 0
        while k ≤ m-1 and P[m-1-k] == T[i-k] do
                k ← k+1
        end while
        if k == m then
            return i-m+1
        else
            i ← i + Table[T[i]]
        end if
    end while
    return -1
}
```

Steps followed by Horspool alg are

1) For the given pattern length (m),



B	2
A	4
R	3
E	1
other	6

Table

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	-
4	2	6	6	1	6	6	6	6	6	6	6	6	6	6	6	6	3	6	6	6	6	6	6	6	6	6

i = m-1 = 21


```
while i ≤ n-1
    k = 0
    while k ≤ m-1 and p[m-1-k] == T[i-k] do
            k++
    if k == m
        return i-m+1
    else
        i ← i + Table[T[i]]
```

Boyer-Moore algorithm

This alg was developed by Robar Boyer & J Strother Moore in 1977. It is considered as the most efficient & widely used alg for pattern matching.

This alg works exactly same as in the Case 1 & Case 2 of Horspool alg. That means, if mismatch occurs at 1st try only, then we are shifting the pattern towards its right by the shift values of the mismatch character ~~with~~ retrieved from the shift table.

That means, this Boyer-Moore alg also uses the same shift table which was used by Horspool alg.

Boyer-Moore alg works differently in Case 3 & 4. That means, when mismatch occurs after few (k) no of successful Comparisons.

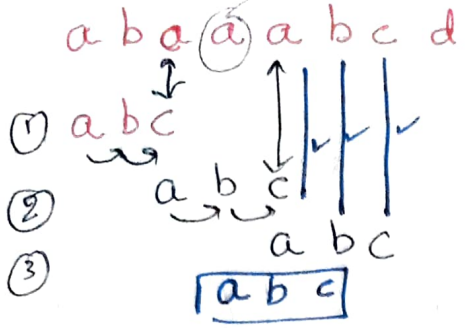
If mismatch occurs after few (k) no of successful Comparisons, shift size is determined by Considering 2 quantities.

1) Bad symbol shift (shift table used by Horspool)
2) Good suffix shift

Bad Symbol shift (shift table @ bad character heuristic)

* after few successful Comparisons, if mismatch occur & if the mismatch character of text (c) is not present in the remaining characters of the pattern, then we are shifting the pattern towards its right by pattern length (m) → Horspool.

But this may lead to miss the chance of finding pattern. Hence, it is called bad symbol shift.



Here ①, a ≠ c, so we shifted the pattern by 2 positions

② again a ≠ c, shift by 2 positions

Now, there may be a chance of missing. We skipped one 'a' i.e., we have not brought the pattern exactly to match 'a' of pattern below 'a' of text. Ac

Actually, whenever the mismatch character is present in the remaining characters of pattern, we need to shift the pattern in such a way that, mismatch character of ~~abbe~~ pattern has to be brought exactly match with the mismatch character of text. abc

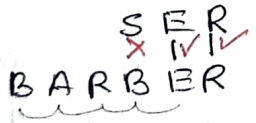
Hence, this table is called as bad symbol shift.

~~Good~~ So, in this case, size of this shift can be computed by the formula $d_1 = t_1(c) - k$

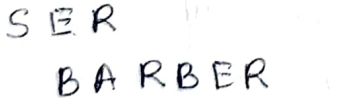
$t_1(c)$ → entry in the precomputed table used by Horspool alg

k → no of matched characters

Ex:-



Here, k=2 & mismatch occurs at 'S'. 'S' is not in the remaining characters of pattern. So, shift by $d_1 = 6-2 = 4$



Same formula, $t_1(c) - k$, is used when mismatching character present in the remaining characters of the pattern.

$d_1 = t_1(A) - k$

$= 4 - 2 = 2$

~~B A R B E R~~

Sometimes $t_1(c) - K$ gives 0 (or) -ve no's.

That means, $t_1(c) - K \leq 0$, at

At this time, it is not possible to shift the pattern 0 times (or) -ve no of times.

So, in such cases, follow brute force method of shifting i.e., shift the pattern by one position.

$$\therefore \boxed{d_1 = \max\{t_1(c) - K, 1\}}$$

That means, if $(t_1(c) - K) \leq 0$ then shift by 1 position.

Good suffix shift

The second type of shift is guided again a mismatch occurs after K no of successful comparisons & the mismatch character is present in the remaining characters of the pattern.

The no of characters matched before mismatch is referred as suffix of size K & denoted as suff(K).

This type of shift we called as good suffix shift.

Assume K no of successful comparison is over & after that mismatch occurs i.e, mismatch occurs after suff(K).

Suppose, if there is another occurrence of suff(K) in the pattern but not preceded by the same character as in its last occurrence.

In this case, we can shift the pattern by the distance d_2 betn such a 2nd rightmost occurrence of suff(K) & its rightmost occurrence.

For ex, ABCBAB → pattern

A B C B A B
1 2

assume 1 is character is matched i.e, B & then mismatch occur at 'A'. ∴ K=1

Now, check in the pattern for another occurrence of suff(K) i.e, check whether 'B' present in the remaining characters of the pattern, & it's distance to the rightmost occurrence of suff(K) (i.e, last character of pattern (last character of 'k' no of matched character))

In the above ex, $d_2 = 2$ for K=1.

Similarly, assume 2 characters has been matched (i.e, K=2) & then mismatch occurs

A B C B A B
1 2 3 4

K=2
suff(K) = AB

Since another occurrence of suffix(K) i.e, AB is present in the remaining characters of pattern, & it is at a distance, $d_2 = 4$

K	pattern	d_2
1	ABCBAB	2
2	ABCBAB	4

Suppose, if there is no other occurrence of suff(K) not preceded by the same character as in its last occurrence, then what to do?

In such a cases, shift the pattern by its length m.

Ex:- ..... S B A B ..... (K=3)
* | | |
D B C B A B → (1)

D B C B A B

Best shifting the pattern by its entire length when there is no other occurrences of suff(K) not preceded by the same character as in its last occurrence is not always correct.

For ex; pattern → A B C B A B & K=3, & shifting by 6 could miss a matching substring that starts with text's AB aligned with the last 2 characters of the pattern.

```
. . . . . - S B A B C B A B - - - -
            * | | |
            A B C B A B
```

→ (2)

A B C B A B

Shifting by 6 is correct in case (1), but not (2) ∴ case (2) has same substring AB as its prefix (beginning part of the pattern) & as its suffix (ending part of the pattern).

To avoid such an erroneous shift based on a suffix of size k, for which there is no other occurrence in the pattern not preceded by the same character as in its last occurrence, we need to find the longest prefix of size $l < k$ (ಕ್ಕಿಂತ ಉದ್ದ prefix ಸಿಕ್ಕರೆ ಉದ್ದ, ಆದ್ರೆ ಸರಿಯಾದ ಉದ್ದ ಅಲ್ಲ. ಉದಾ, K=5, ನಾವು 5 character prefix ತಗೊಂಡ್ರು, ಅದ್ರಲ್ಲಿ ಸರಿಯಾದ ಉದ್ದ, ಉದ್ದ - 1). that matches the suffix of the same size l.

If such a prefix is exists, the shift size d_2 is computed as the distance bet^n this prefix & the corresponding suffix; otherwise, d_2 is set to m, pattern length.

K	pattern	d_2
1	ABCBAB	2
2	ABCBAB	4
3	ABCBAB	4
4	ABCBAB	4
5	ABCBAB	4

Summary of steps

1) Construct the bad symbol shift table. (Same as in Horspool)
2) Using the pattern, construct the good suffix table
3) align the pattern against the beginning of the text
4) Repeat the following until either a pattern is found (or) pattern reaches beyond the last character of the text.
   → Starting with last character in the pattern, compare the corresponding characters in the pattern & the text until either all 'm' character pairs are matched (then stop) (or) a mismatching pair is encountered after $k \geq 0$ character pairs are matched successfully.
   → In the latter case, retrieve the entry $t_1(c)$ from bad symbol table.
   → If $k > 0$, also retrieve the corresponding d_2 entry from the good suffix table.
   Shift the pattern to the right by the no of positions computed by the formula

$$d = \begin{cases} d_1 & \text{if } k = 0 \\ \max\{d_1, d_2\} & \text{if } k > 0 \end{cases}$$

where $d_1 = \max\{t_1(c) - k, 1\}$

MADHUSUDHAN M.V.
Dept. of CSE

Problem:

BESS_KNEW_ABOUT_BAOBABS ⟶ Text

BAOBAB ⟶ pattern

Solution:

c	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	_
$t_1(c)$	1	2	6	6	6	6	6	6	6	6	6	6	6	6	3	6	6	6	6	6	6	6	6	6	6	6	6

Bad symbol table

k	pattern	d_2
1	BAOB̄AB	2
2	B̄AOBAB	5
3	B̄AOBAB	5
4	B̄AOBAB	5
5	B̄AOBAB	5

Good suffix table

BAOBAB

2 → 1

↓ 3

③ Here, K=1

$d_1 = \max\{t_1(-) - 1, 1\}$

$= \max\{6 - 1, 1\}$

$d_1 = 5$

$d_2 = 2$

So, shift by 5

BESS_KNEW_ABOUT_BAOBABS

① BAOBAB

$d_1 = \max\{t_1(K) - 0, 1\}$

$= \max\{6 - 0, 1\}$

$d_1 = 6$

So shift by 6

② BAOBAB

K=2

$d_1 = \max\{t_1(-) - 2, 1\}$

$= \max\{6 - 2, 1\}$

$d_1 = 4$

$d_2 = 5$

So, shift by 5

BAOBAB ③

BAOBAB

Here, we shifted 3 times & Compared 12 times